



**Федеральное государственное бюджетное
образовательное**

учреждение высшего образования

**«Национальный исследовательский университет
«МЭИ»**

Институт: **ИВТИ**

Кафедра: **ВМСС**

Направление подготовки:

09.03.01

Информатика

и

вычислительная техника

**Отчёт по дисциплине “Технология программирования”
на тему “Разработка приложения для администрирования
фитнессклуба”**

Работу выполнил

Студент Андрюкова Е.

Группа А-08-24

Работу прове

Москва, 2025

Оглавление

Введение.....	3
1. Техническое задание.....	4
2. Разработка диаграммы классов.....	5
3. Разработка и реализация программы на языке C++	11
4. Разработка и реализация программы на языке C#.....	22
Заключение	36

Введение

В современном мире индустрия здорового образа жизни и фитнеса переживает бурный рост. Эффективное управление фитнес-клубом требует оперативной обработки больших объемов данных: ведения расписания тренеров, учета клиентской базы, регистрации тренировок и сбора обратной связи. Ручное ведение этих процессов приводит к ошибкам, накладкам в расписании и снижению качества обслуживания.

Автоматизация деятельности фитнес-клуба позволяет систематизировать информационные потоки, исключить «человеческий фактор» при бронировании времени и обеспечить прозрачность взаимодействия между клиентом, тренером и администратором.

Актуальность разработки программного обеспечения для менеджера фитнес-клуба обусловлена необходимостью создания удобного инструмента, объединяющего функционал записи на тренировки, управления статусами заказов и ведения финансовой статистики в единой информационной среде.

Цель работы: разработка оконного приложения для автоматизации процессов фитнес-клуба (управление записями и пользователями).

Задачи работы:

1. Анализ предметной области и формирование требований к системе учета тренировок.
2. Проектирование архитектуры приложения с использованием принципов объектно-ориентированного программирования (ООП), включая наследование, полиморфизм и инкапсуляцию.
3. Реализация консольной версии приложения на языке C++ с использованием классов для представления ролей (Клиент, Тренер, Администратор) и сущностей (Заказ).
4. Разработка оконного приложения на языке C# (Windows Forms) с улучшенным пользовательским интерфейсом и системой сохранения данных.
5. Реализация алгоритмов бронирования временных слотов, подтверждения заказов и расчета рейтинга тренеров.
6. Проведение сравнительного анализа двух реализаций.

Разрабатываемая программа обеспечит комплексное управление ресурсами фитнес-клуба, начиная от регистрации новых пользователей и заканчивая получением обратной связи по проведенным тренировкам.

1. Техническое задание

Подробное описание предметной области

Разрабатываемое программное обеспечение «FitManager» представляет собой информационную систему для автоматизации управления рабочими процессами фитнес-клуба. Система предназначена для организации взаимодействия трех категорий пользователей: Клиентов, Тренеров и Администраторов.

Основной сущностью системы является «Тренировка» (Заказ). Жизненный цикл заказа включает следующие последовательные этапы:

1. **Инициализация:** Клиент выбирает направление тренировки (например, «Похудение», «Йога», «Кардио»), выбирает тренера из списка доступных специалистов и указывает желаемую дату и время. Заказ создается со статусом «Ожидает» (Pending).
2. **Обработка:** Тренер (или Администратор) просматривает список поступивших заявок. При наличии свободного времени в расписании заявка подтверждается, статус меняется на «Подтверждено» (Confirmed), а временной слот бронируется, становясь недоступным для других клиентов. Заявка также может быть отклонена, при этом слот освобождается.
3. **Выполнение:** После фактического проведения занятия Тренер отмечает заказ как выполненный, переводя его в статус «Завершено» (Completed).
4. **Оценка:** Клиент получает возможность оставить отзыв о проведенной тренировке (рейтинг от 1 до 5). После этого заказ переходит в финальный архивный статус «Закрыт» (Closed), а средний рейтинг тренера автоматически пересчитывается.

Ограничения по данным программы

1. Учетные записи и безопасность

- **Логин:** Должен быть уникальным в рамках системы. При регистрации система проверяет наличие пользователя с таким же логином.

- **Пароль:** Должен содержать не менее 4 символов (для обеспечения базовой безопасности).

2. Финансовые и рейтинговые показатели

- **Рейтинг тренера:** Рассчитывается динамически как среднее арифметическое всех полученных оценок. Диапазон значений строго ограничен от 1.0 до 5.0.
- **Стоимость тренировки:** Выражается положительным числом. Базовая стоимость зависит от квалификации тренера.

3. Логические ограничения расписания

- Запись на тренировку возможна только на те временные слоты, которые не заняты другими подтвержденными заказами .
- Изменение статуса заказа возможно только в соответствии с установленным жизненным циклом (например, нельзя оценить тренировку, которая еще не была проведена).
- Один клиент может иметь неограниченное количество заказов в истории.

4. Требования к реализации данных

- Приложение должно обеспечивать валидацию вводимых данных (защита от ввода букв вместо цифр при выборе меню или указании рейтинга).
- В графической версии (C#) данные о пользователях и заказах должны сохраняться в файлы формата JSON для обеспечения сохранения состояния между сеансами работы.

2. Разработка диаграммы классов

Была разработана диаграмма классов.

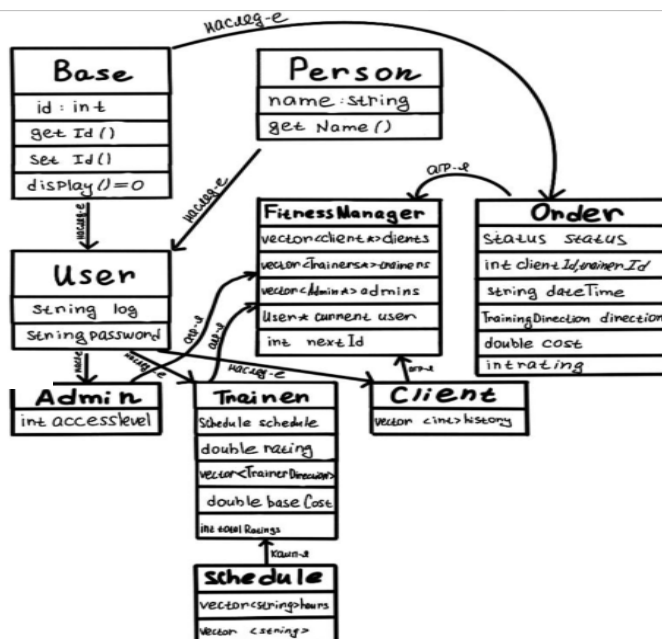


Рис. 1 Диаграмма классов.

Абстрактный класс Base

Назначение: Базовый класс для всех сущностей системы, имеющих уникальный идентификатор.

Табл. 1 Описание полей и методов класса Base

Имя	Модификатор доступа	Описание
Описание полей		
int id	private	Уникальный идентификатор объекта
Описание методов		
Base()	public	Конструктор по умолчанию
Base(int i)	public	Конструктор с параметром ID
virtual ~Base()	public	Виртуальный деструктор
int getId() const	public	Геттер идентификатора

virtual void display() const	public	Чисто виртуальный метод вывода информации
------------------------------	--------	---

Абстрактный класс User

Назначение: Базовый класс пользователя. Реализует множественное наследование от Base и Person, объединяя функционал идентификации и личных данных.

Табл. 2 Описание полей и методов класса User

Имя	Модификатор доступа	Описание
Описание полей		
string login	private	Логин для входа в систему
string password	private	Пароль пользователя
Описание методов		
bool auth(string l, string p)	public	Метод аутентификации (проверка логина/пароля)
virtual string getRole() const	public	Абстрактный метод получения роли пользователя
const string& getLogin() const	public	Геттер логина

Класс Client

Назначение: Класс, описывающий клиента фитнес-клуба. Наследуется от User.

Табл. 3 Описание полей и методов класса Client

Имя	Модификатор доступа	Описание
Описание полей		
vector<int> history	private	Список идентификаторов заказов клиента
Описание методов		
Client()	public	Конструктор
void addOrderId(int oid)	public	Добавление заказа в историю
int getHistoryCount() const	public	Получение количества тренировок
void display() const	public	Переопределенный метод вывода информации

Класс Trainer

Назначение: Класс тренера. Содержит расписание, рейтинг и специализации.

Табл. 4 Описание полей и методов класса Trainer

Имя	Модификатор доступа	Описание
Описание полей		

Schedule schedule	private	Объект расписания (Композиция)
double rating	private	Текущий рейтинг тренера
vector<TrainingDirection> specs	private	Список специализаций
Описание методов		
bool book(string date, string time)	public	Бронирование слота в расписании
vector<string> getAvailableSlots()	public	Получение свободных слотов
void addRating(int newRating)	public	Пересчет рейтинга
double priceFor(TrainingDirection)	public	Расчет стоимости тренировки

Класс Order

Назначение: Класс заказа (тренировки). Хранит информацию о статусе, участниках и стоимости.

Табл. 5 Описание полей и методов класса Order

Имя	Модификатор доступа	Описание
Описание полей		
Status status	private	Текущий статус заказа
int clientId	private	ID клиента
int trainerId	private	ID тренера
string dateTime	private	Дата и время проведения
Описание методов		
Order(int id, ...)	public	Конструктор с параметрами

void changeStatus(Status s)	public	Изменение статуса заказа
bool addReview(int r)	public	Добавление оценки
void displayDetailed() const	public	Вывод полной информации о заказе

Класс **FitnessManager**

Назначение: Управляющий класс . Хранит списки объектов и реализует основную логику.

Табл. 6 Описание полей и методов класса FitnessManager

Имя	Модификатор доступа	Описание
Описание полей		
vector<Client*> clients	private	Список всех клиентов
vector<Trainer*> trainers	private	Список всех тренеров
vector<Order*> orders	private	Список всех заказов
User* currentUser	private	Указатель на текущего пользователя
Описание методов		
void run()	public	Запуск главного цикла приложения
bool login(string l, string p)	public	Вход в систему
void registerClient()	public	Регистрация нового клиента
bool bookTraining(...)	public	Создание нового заказа

3. Разработка и реализация программы на языке C++

3.1. Разработка пользовательского интерфейса

1. Главное меню

Назначение: Предоставляет пользователю выбор между входом в систему и регистрацией нового аккаунта.

При запуске приложения отображается приветственное сообщение и список доступных действий (Рис. 2):

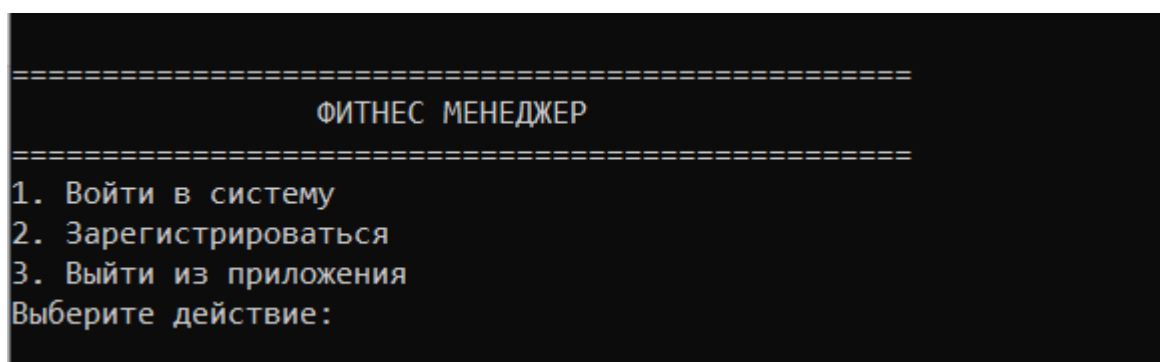


Рис. 2 Главное меню программы

2. Регистрация

Назначение: Создание учетной записи для нового клиента.

При выборе пункта регистрации система запрашивает уникальный логин, пароль и ФИО пользователя. После успешного ввода данных клиент добавляется в базу (Рис. 3):

```
=====
                        ФИТНЕС МЕНЕДЖЕР
=====
1. Войти в систему
2. Зарегистрироваться
3. Выйти из приложения
Выберите действие: 2

=== Регистрация нового клиента ===
Логин: client1
Пароль: 1234
ФИО: Иван Иванов
Регистрация выполнена!
```

Рис. 3 Процесс регистрации нового клиента

3. Создание заказа (Запись на тренировку)

Назначение: Бронирование времени у тренера.

Пользователь выбирает направление тренировки и тренера. Система проверяет расписание выбранного специалиста и предлагает свободные временные слоты на указанную дату. После выбора времени создается заказ (Рис. 4):

```
=====
                        ФИТНЕС МЕНЕДЖЕР
=====

1. Войти в систему
2. Зарегистрироваться
3. Выйти из приложения
Выберите действие: 1
Логин: client1
Пароль: 1234

Успешный вход! Добро пожаловать, !

=== МЕНЮ КЛИЕНТА ===
1. Записаться на тренировку
2. Посмотреть мои тренировки
3. Оставить оценку
4. Показать всех тренеров
5. Выйти из аккаунта
Выберите действие: 1

=== Запись на тренировку ===
1.Похудение 2.Набор массы 3.Йога 4.Кардио
Выберите направление (1-4): 1

=== Доступные тренеры ===
1. ID:1 | | Роль: Тренер | Рейтинг: 4.8 | Оценок: 10 | Базовая цена: 1500
   Специализации: Похудение, Кардио
Тренер (номер): 1
Дата (ДД.ММ.ГГГГ): 20.12.2025
Доступное время: 1.09:00 2.10:00 3.11:00 4.12:00 5.14:00 6.15:00 7.16:00 8.17:00
Выберите время (номер): 1
Бронь создана: 20.12.2025 09:00
```

Рис. 4 Процесс записи на тренировку

4. Просмотр истории

Назначение: Отображение списка текущих и завершенных тренировок.

Клиент может просмотреть детали своих заказов, включая статус, дату, тренера и стоимость (Рис. 5):

```
=== МЕНЮ КЛИЕНТА ===
1. Записаться на тренировку
2. Посмотреть мои тренировки
3. Оставить оценку
4. Показать всех тренеров
5. Выйти из аккаунта
Выберите действие: 2

=== Ваши тренировки ===
Заказ#10 | 20.12.2025 09:00 | Похудение | 1500 руб. | статус=ожид.

=== МЕНЮ КЛИЕНТА ===
1. Записаться на тренировку
2. Посмотреть мои тренировки
3. Оставить оценку
4. Показать всех тренеров
5. Выйти из аккаунта
Выберите действие:
```

Рис. 5 История тренировок клиента

5. Интерфейс Тренера

Назначение: Управление расписанием и подтверждение тренировок.

Тренер имеет доступ к своему личному кабинету, где может просматривать поступившие заявки, свое расписание на конкретную дату и подтверждать или отменять записи (Рис. 6):

```
=== МЕНЮ ТРЕНЕРА ===
1. Просмотреть мои заказы
2. Мое расписание
3. Подтвердить/отказать заказ
4. Завершить тренировку
5. Выйти из аккаунта
Выберите действие: 1

=== Заказы тренера ===
Заказ#7 | 01.11.2025 10:00 | Похудение | 1500 руб. | статус=ожид.

Заказ#10 | 20.12.2025 09:00 | Похудение | 1500 руб. | статус=ожид.

=== МЕНЮ ТРЕНЕРА ===
1. Просмотреть мои заказы
2. Мое расписание
3. Подтвердить/отказать заказ
4. Завершить тренировку
5. Выйти из аккаунта
Выберите действие: 2
Дата (ДД.ММ.ГГГГ): 11.12.2025
Свободное время: 09:00 10:00 11:00 12:00 14:00 15:00 16:00 17:00
```

Рис. 6 Меню тренера

6. Интерфейс Администратора

Назначение: Полный контроль над системой.

Администратор обладает правами просмотра общей базы данных всех клиентов, тренеров и заказов, а также сводной статистики по работе клуба (Рис. 7):

```

=== МЕНЮ АДМИНИСТРАТОРА ===
1. Просмотреть всех пользователей
2. Просмотреть все заказы
3. Информация о системе
4. Выйти из аккаунта
Выберите действие: 1

=== Все пользователи системы ===
Клиенты (3):
  ID:4 |   | Роль: Клиент | Заказов: 1
  ID:5 |   | Роль: Клиент | Заказов: 1
  ID:9 |   | Роль: Клиент | Заказов: 1

Тренеры (3):
  ID:1 |   | Роль: Тренер | Рейтинг: 4.8 | Оценок: 10 | Базовая цена: 1500
    Специализации: Похудение, Кардио
  ID:2 |   | Роль: Тренер | Рейтинг: 4.9 | Оценок: 10 | Базовая цена: 2000
    Специализации: Набор массы, Йога
  ID:3 |   | Роль: Тренер | Рейтинг: 4.7 | Оценок: 10 | Базовая цена: 1200
    Специализации: Йога, Кардио

Администраторы (1):
  ID:6 |   | Роль: Админ | Уровень доступа: 3

=== МЕНЮ АДМИНИСТРАТОРА ===
1. Просмотреть всех пользователей
2. Просмотреть все заказы
3. Информация о системе
4. Выйти из аккаунта
Выберите действие: 3

=== Информация о системе ===
Всего пользователей: 7
Всего заказов: 3
Следующий ID: 11
ID:6 |   | Роль: Админ | Уровень доступа: 3

```

Рис. 7 Панель администратора

3.2. Описание программы

3.2.1 Общее описание программы

Разработанное программное обеспечение «FitManager» представляет собой консольное приложение на языке C++, моделирующее работу системы управления фитнес-клубом. Программа построена на строгих принципах объектно-ориентированного программирования.

Система обеспечивает полный цикл работы менеджера: от регистрации новых клиентов до получения обратной связи по проведенным тренировкам. Взаимодействие с пользователем осуществляется через текстовое консольное меню, защищенное от некорректного ввода данных.

Архитектура системы включает иерархию классов пользователей (User, Client, Trainer, Admin), наследуемую от базовых классов Base и Person. Для хранения списков объектов используются динамические контейнеры стандартной библиотеки шаблонов (STL) `std::vector`.

Ключевые особенности реализации на C++:

Использование множественного наследования для формирования сущности пользователя.

Применение полиморфизма для обработки разнородных объектов (клиентов, тренеров) через указатели на базовый класс.

Реализация композиции (класс Trainer жестко владеет объектом Schedule) и агрегации (класс Client хранит историю заказов).

Обработка исключительных ситуаций при вводе данных.

3.2.2 Реализация по чек-листу

Пункт чек-листа	Пример реализации в коде
Класс	<code>class Order, class User, class Trainer</code>
Геттер / Сеттер	<code>getId(), setLogin(), getRole()</code>
Конструктор с параметрами	<code>User(int id, const string& lg ...)</code>
Конструктор копирования	<code>User(const User& u)</code>
Конструктор без параметров	<code>User(), Schedule()</code>
Деструктор / Виртуальный	<code>~User(), virtual ~Base() {}</code>
Дружественные функции	<code>friend ostream& operator<<</code>
Дружественный класс	<code>friend class AuditOrder</code>
Перегрузка операторов	<code>Schedule& operator++(), operator+</code>
Перегрузка ввода/вывода	<code>friend ostream& operator<<</code>
Одиночное наследование	<code>class Admin : public User</code>
Множественное наследование	<code>class User : public Base, virtual public Person</code>
Агрегация	<code>Client содержит vector<int> history</code>
Композиция	<code>Trainer содержит Schedule schedule</code>
Абстрактный класс	<code>class Base, class User</code>
Чисто виртуальная функция	<code>virtual void display() const = 0;</code>

Табл. 7 Таблица соответствия чек-листа и разработанной программы

3.3. Тестирование программы

Объект испытаний – разработанное консольное приложение «FitManager» для управления фитнес-клубом.

Цель испытаний:

- Проверка корректности работы всех функций программы согласно техническому заданию.
- Выявление ошибок и несоответствий в логике работы.
- Проверка обработки граничных условий и исключительных ситуаций.
- Оценка надежности системы при некорректном вводе данных.

Средства испытаний: среда разработки Visual Studio 2022, компилятор MSVC, командная строка Windows.

Порядок и методы испытаний: Используется метод функционального тестирования («черный ящик»): проверка работы программы без анализа исходного кода, основываясь только на входных и выходных данных.

Тестирование проводится в три этапа:

1. **Нормальные условия** — штатные сценарии работы (регистрация, запись).
2. **Граничные условия** — проверка крайних значений меню и дат.
3. **Исключительные условия** — ввод заведомо некорректных данных (буквы вместо цифр, несуществующие даты).

Тестирование в нормальных условиях

Тест 1.1. Регистрация нового пользователя.

Ожидаемый результат: Система запрашивает данные, проверяет уникальность логина и успешно сохраняет нового пользователя в векторе. **Фактический результат:** Регистрация проходит успешно, программа переходит в главное меню (см. Рис. 3 в п. 3.1).

Тест 1.2. Создание заказа на тренировку.

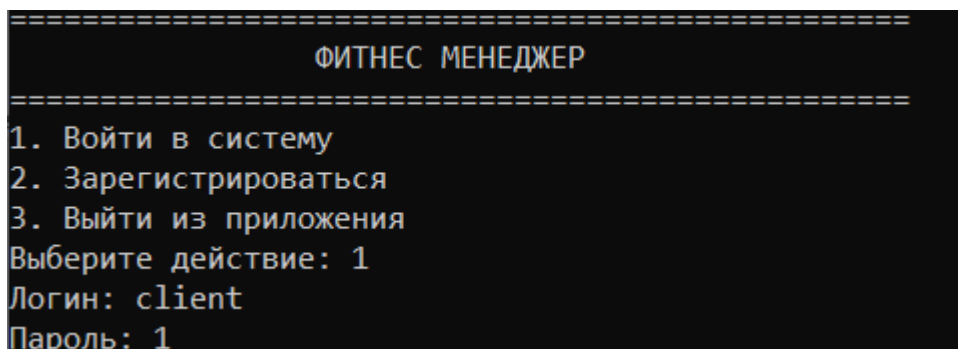
Ожидаемый результат: Пользователь выбирает направление и тренера. Система отображает свободные слоты. После выбора времени заказ сохраняется.

Фактический результат: Заказ успешно создается и отображается в истории клиента (см. Рис. 4 и Рис. 5 в п. 3.1).

Тестирование в граничных условиях

Тест 2.1. Выбор первого пункта меню (Минимальное значение).

Ожидаемый результат: При вводе «1» система корректно переходит к процедуре авторизации.



```
=====
                        ФИТНЕС МЕНЕДЖЕР
=====
1. Войти в систему
2. Зарегистрироваться
3. Выйти из приложения
Выберите действие: 1
Логин: client
Пароль: 1
```

(Рис. 8 Выбор первого пункта меню **Фактический результат:** Переход выполнен корректно.

Тест 2.2. Выбор пункта «Выход» (Максимальное значение) **Ожидаемый результат:** При выборе пункта выхода (цифра «3») программа должна завершить выполнение всех функций, очистить память и закрыть окно консоли.

Фактический результат: При вводе команды выхода приложение завершает работу штатно, окно консоли закрывается, сообщений о критических ошибках системы не возникает.

Тестирование в исключительных условиях

Тест 3.1. Некорректный ввод данных в меню.

Ожидаемый результат: При вводе строковых символов (букв) вместо ожидаемого числа программа должна вывести сообщение об ошибке, очистить поток ввода и повторить запрос, не допуская аварийного закрытия.

```
=====
                        ФИТНЕС МЕНЕДЖЕР
=====
1. Войти в систему
2. Зарегистрироваться
3. Выйти из приложения
Выберите действие: вусыв
Ошибка! Введите число от 1 до 3
Выберите действие: █
```

Рис. 9 Обработка нечислового ввода

Фактический результат: Система перехватывает ошибку ввода, выводит предупреждение и позволяет повторить попытку.

Тест 3.2. Ввод некорректной даты.

Ожидаемый результат: Система должна валидировать формат даты. Ввод несуществующей даты (например, 32-е число) или даты в прошлом должен блокироваться.

```
Дата (ДД.ММ.ГГГГ): 32.10.2025
Ошибка! Некорректная дата. Год должен быть от 2025 до 2030
```

Рис. 10 Валидация даты **Фактический результат:** Некорректная дата отклоняется, заказ не создается.

Результаты тестирования программы

Итоговые результаты испытаний консольной версии представлены в таблице.

Табл. 8 Результаты тестирования

Номер теста	Условия тестирования	Результат
-------------	----------------------	-----------

1.1	Нормальные	Успешно
1.2	Нормальные	Успешно
2.1	Граничные	Успешно
2.2	Граничные	Успешно
3.1	Исключительные	Успешно
3.2	Исключительные	Успешно

4. Разработка и реализация программы на языке C#

4.1. Разработка пользовательского интерфейса

Реализация на языке C# выполнена с использованием технологии Windows Forms, что обеспечивает интуитивно понятное графическое взаимодействие с пользователем. Элементы управления (кнопки, текстовые поля, таблицы) предоставляют визуальную обратную связь.

Разработанная диаграмма классов C#:

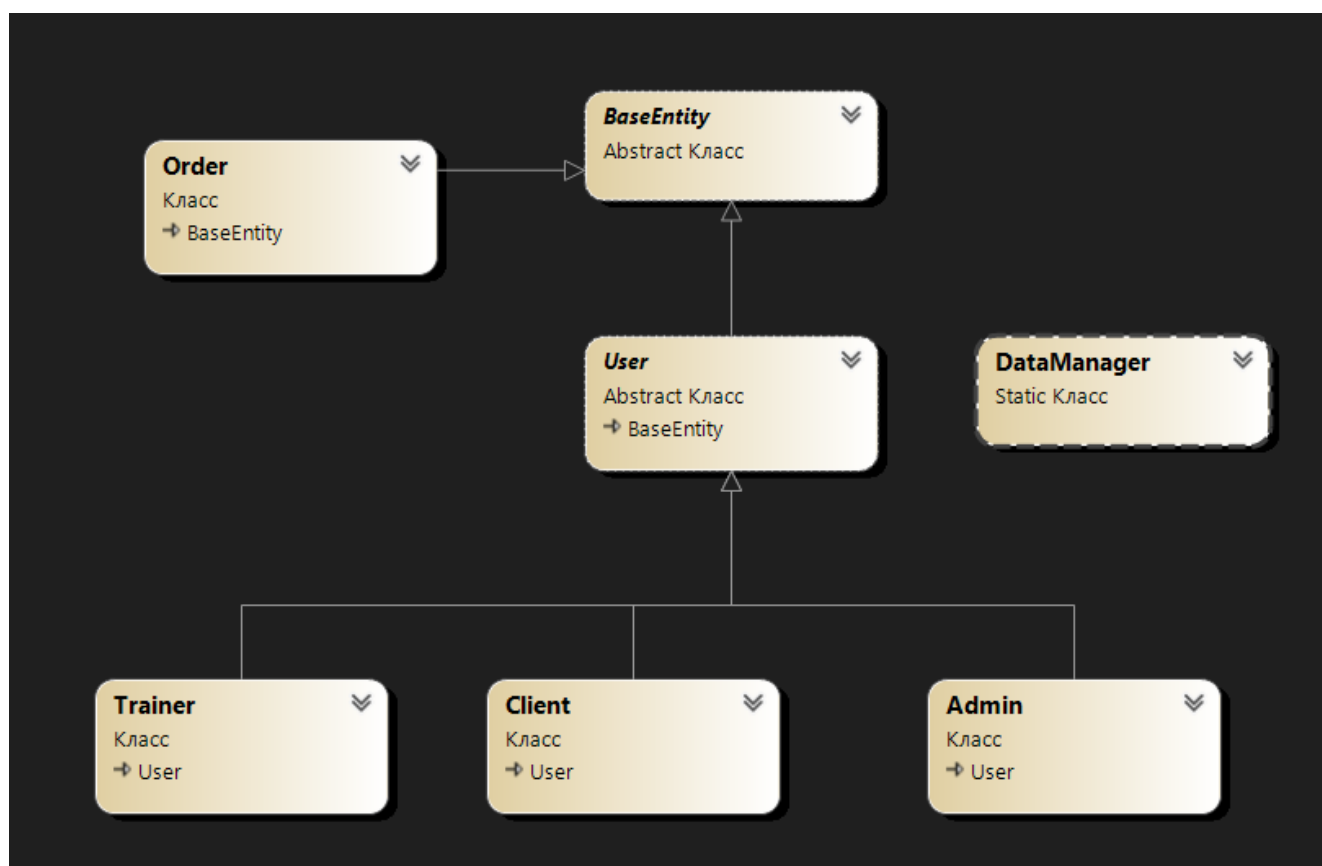


Рис. 11 Диаграмма классов.

Окно авторизации При запуске приложения открывается форма входа. Пользователь вводит логин и пароль. Для безопасности символы пароля скрыты. Также предусмотрена кнопка перехода к регистрации новых клиентов (Рис. 12).

The screenshot shows a simple login window with a title bar containing standard Windows window controls. The window has a light gray background. It contains two text input fields: the first is labeled 'Логин' (Login) and the second is labeled 'Пароль' (Password). Below the password field, there are two buttons: 'Войти' (Login) and 'Зарегистрироваться' (Register). The buttons are rectangular with a light gray background and a thin border.

Рис. 12 Окно авторизации

Регистрация нового пользователя

Форма регистрации позволяет создать новую учетную запись. Система проверяет заполненность всех полей и уникальность логина в базе данных перед сохранением (Рис. 13)

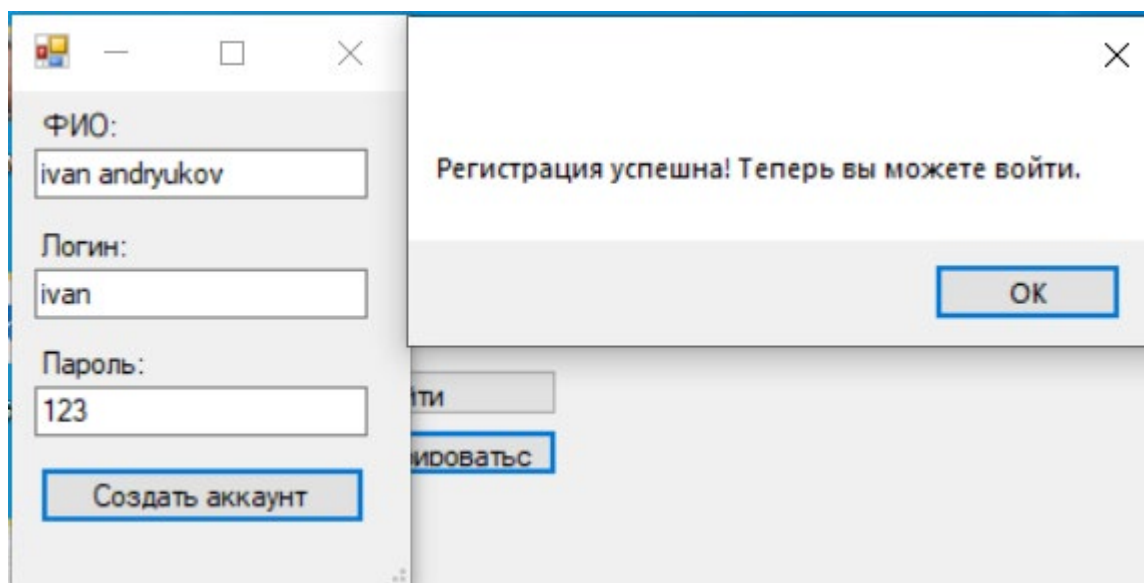


Рис. 13 Форма регистрации

Рабочее место Клиента

Основное окно клиента содержит вкладки для навигации. На вкладке «Запись» реализована динамическая фильтрация: при выборе направления в таблице отображаются только соответствующие тренеры. Использование календаря (DateTimePicker) исключает ошибки формата даты (Рис. 14).

ClientForm

Запись на тренировку | Мои тренировки

Выберите направление

WeightLoss

Доступные тренеры

	Rating	BaseCost	Name
▶	3,5	1500	Иван Петров

Дата тренировки:

12.12.2025

Время:

09:00

ЗАПИСАТЬСЯ

Рис. 14 Интерфейс записи на тренировку

Панель Администратора

Администратор имеет доступ к полному реестру пользователей и заказов. Данные представлены в виде таблиц (DataGridView), что позволяет наглядно оценивать загруженность клуба и просматривать статусы всех заказов (Рис. 15).

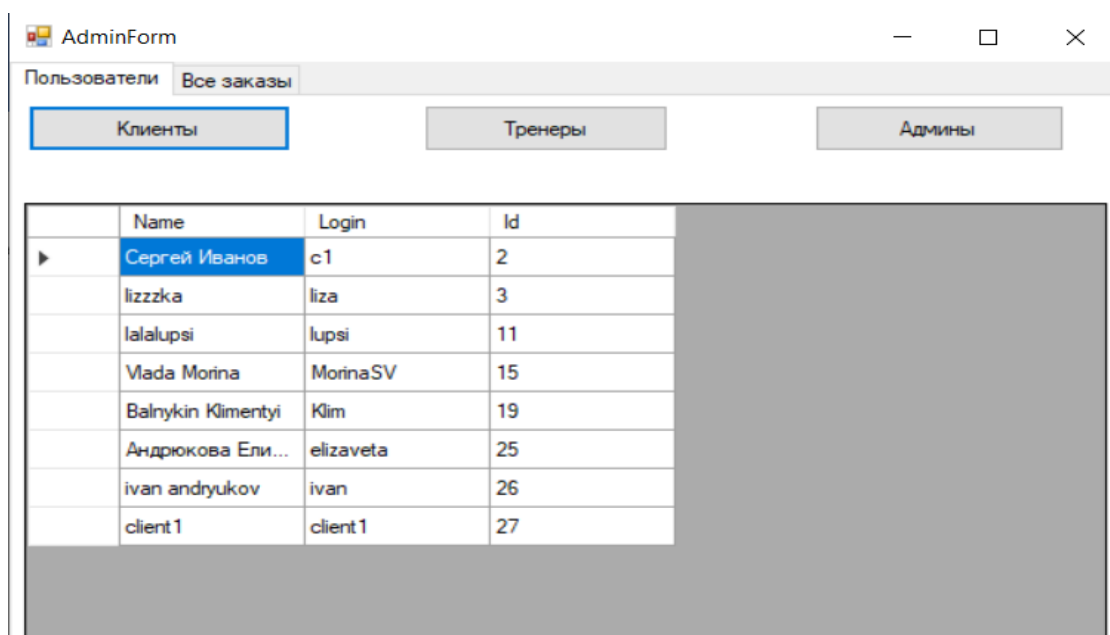


Рис. 15 Интерфейс администратора

4.2. Описание программы

4.2.1 Общее описание Программа «FitnessManagerApp» — это оконное приложение, разработанное на платформе .NET. В отличие от консольной версии, здесь реализована событийно-ориентированная модель (Event-driven programming): программа ожидает действий пользователя (нажатия, выбор из списка) и реагирует на них.

Важной особенностью версии на C# является система сохранения данных (персистентность). Вся информация о клиентах, тренерах и заказах сериализуется в формат JSON и сохраняется в локальные файлы. Это позволяет восстанавливать состояние системы после перезапуска.

4.2.2 Реализация по чек-листу

В таблице ниже приведен анализ реализации принципов ООП в версии C#. Особенности языка (например, отсутствие множественного наследования классов) учтены в реализации.

Табл. 9 Таблица соответствия чек-листа и реализации на C#

Пункты чек-листа	Реализация в C#
Класс	public class Client, public class Order
Геттер / Сеттер	Автосвойства: public string Login { get; set; }
Конструктор:	
- с параметрами	public Order(...) { ... } (инициализация объекта)
- копирования	Не используется (объекты создаются через new)
- без параметров	public Client() { } (необходим для JSON-сериализации)
- перемещения	- (Управление памятью автоматическое)
Деструктор:	
- Деструктор	- (Используется Garbage Collector)
- Виртуальный деструктор	- (Не требуется в C#)
Дружественные функции	- (Заменены модификатором internal)
Дружественный класс	- (Аналог: internal class внутри сборки)
Перегрузка операторов:	
- через дружественные функции	- (Операторы в C# всегда public static)
- ввода и вывода	Переопределение ToString()
Наследование:	
- одиночное наследование	public class Trainer : User
- множественное наследование	- (Не поддерживается классами в C#)
Композиция объектов:	
- агрегация	DataManager содержит списки List<Client>
- ассоциация	Order хранит идентификаторы ClientId, TrainerId
Раннее и позднее связывание	Виртуальные методы и полиморфизм
Абстрактный класс	public abstract class User
Чисто виртуальная функция	Абстрактный метод public abstract string GetRole();

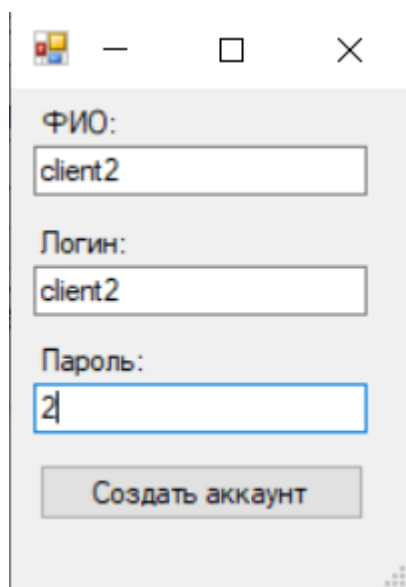
4.3. Тестирование программы

Описание методики тестирования приведено в пункте 3.3. Для графической версии приложения дополнительно проверяется корректность работы элементов управления (кнопок, текстовых полей) и реакция интерфейса на действия пользователя.

Тестирование в нормальных условиях

Тест 1.1. Регистрация и вход в систему.

Ожидаемый результат: При вводе корректных данных новый пользователь успешно сохраняется в файл JSON. После перезапуска приложения возможен вход под этим логином.



The image shows a screenshot of a registration window. The window has a title bar with standard Windows controls (minimize, maximize, close). The form contains three text input fields: 'ФИО:' with 'client2', 'Логин:' with 'client2', and 'Пароль:' with '2'. Below the fields is a button labeled 'Создать аккаунт'. The window has a light gray background and a small icon in the top-left corner.

Рис. 16 Регистрация нового клиента

Фактический результат: Регистрация проходит успешно, данные сохраняются и восстанавливаются после перезагрузки.

Тест 1.2. Создание заказа

Ожидаемый результат: Клиент выбирает направление и тренера. Заказ успешно создается и отображается в списке активных.

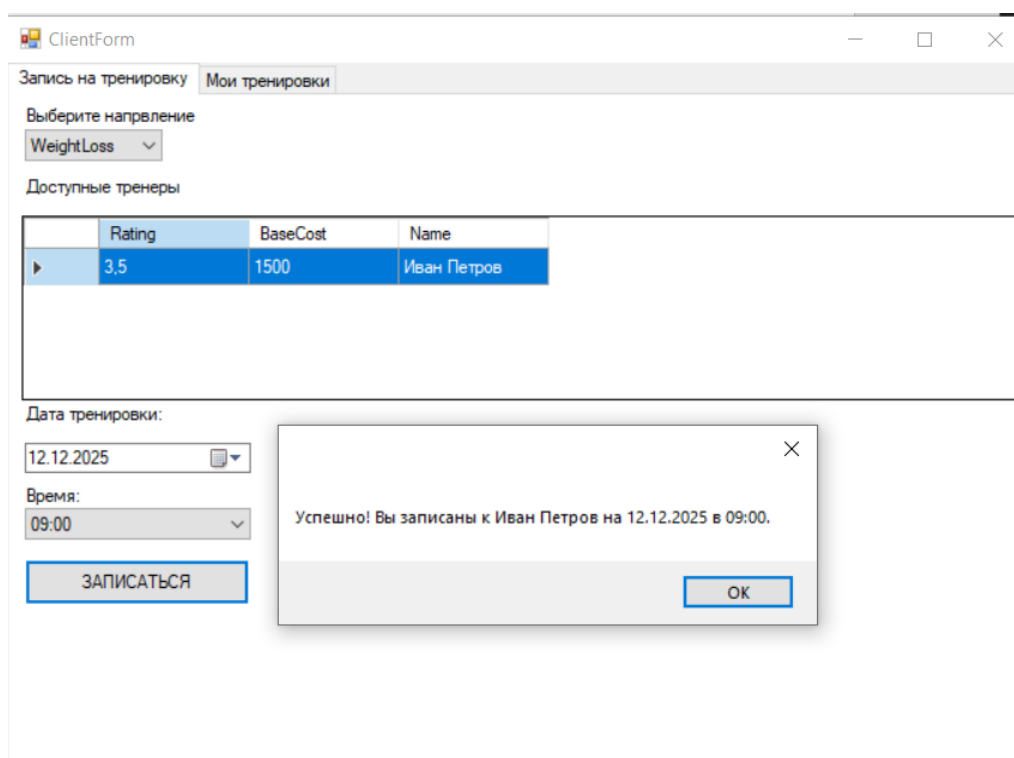


Рис. 17 Успешное создание заказа

Фактический результат: Заказ корректно создается и связывается с выбранным тренером.

Тестирование в граничных условиях

Тест 2.1. Выбор минимального временного слота (Начало рабочего дня)

Ожидаемый результат: Система должна корректно обрабатывать выбор первого доступного значения в списке времени (09:00) и позволять создать запись.

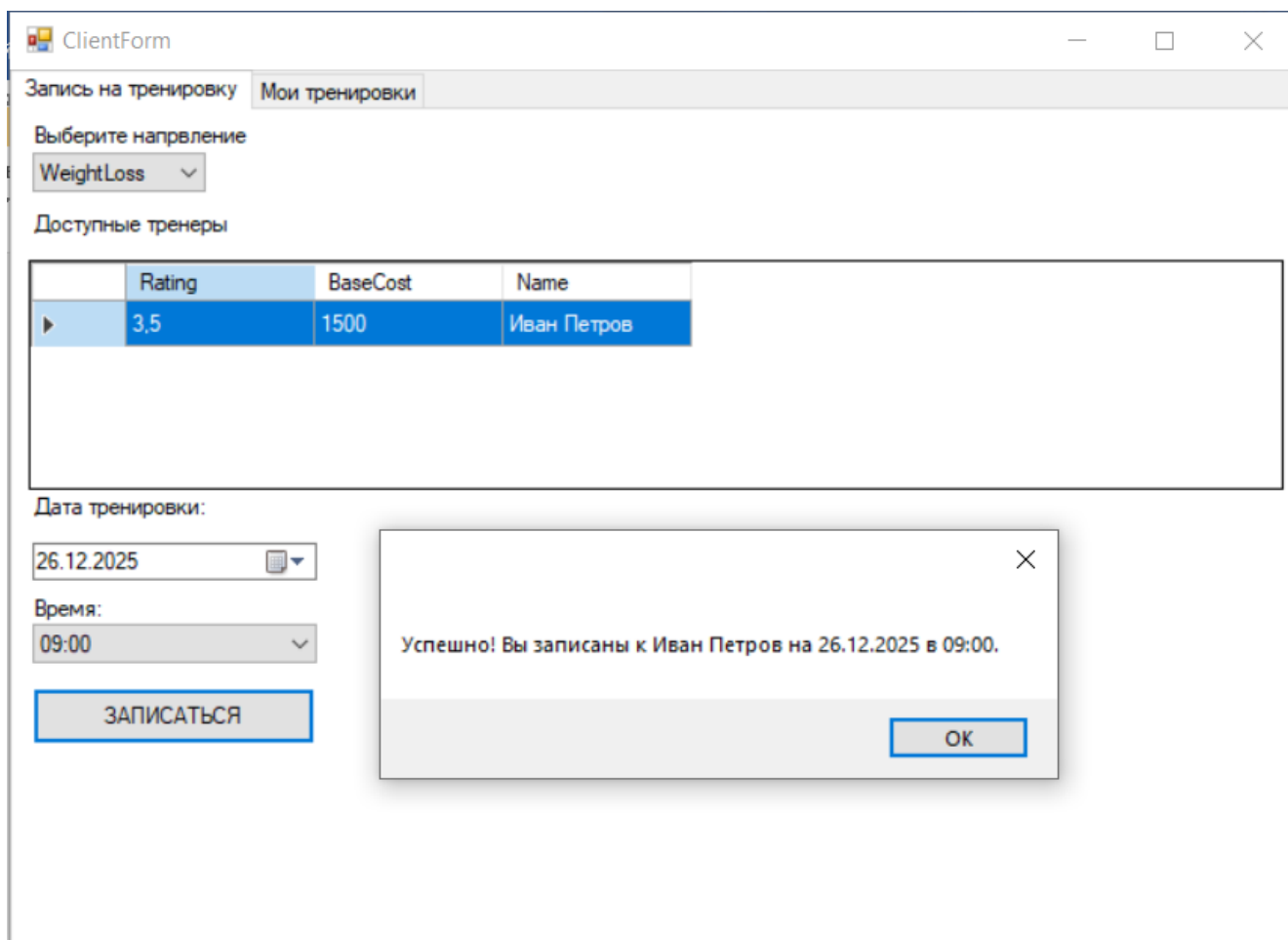


Рис. 18 Выбор первого временного слота

Фактический результат: Граничное значение (открытие смены) обрабатывается корректно.

Тестирование в исключительных условиях

Тест 3.1. Попытка регистрации с пустыми полями.

Ожидаемый результат: При нажатии кнопки «Создать аккаунт» с незаполненными полями система должна вывести предупреждающее сообщение MessageBox и прервать операцию.

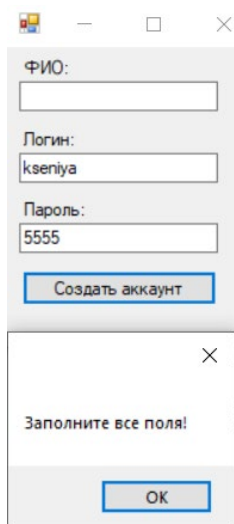


Рис. 19 Ошибка при пустых полях

Фактический результат: Программа корректно валидирует ввод (Рис. 18).

Тест 3.2. Регистрация с существующим логином **Ожидаемый результат:** Система должна проверить наличие логина в базе. Если логин занят (например, «admin»), должно появиться сообщение об ошибке.

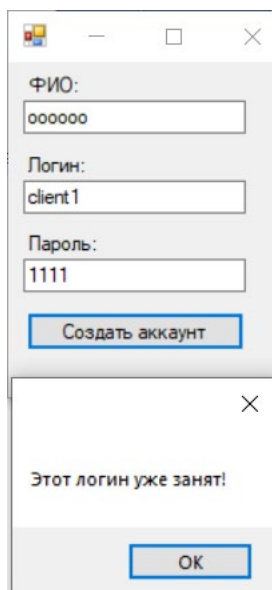


Рис. 20 Защита уникальности логина

Фактический результат: Система предотвращает дублирование пользователей.

Результаты тестирования программы

Результаты испытаний графической версии представлены в таблице.

Табл. 10 Результаты тестирования (C#)

Номер теста	Условия тестирования	Результат
1.1	Нормальные	Успешно
1.2	Нормальные	Успешно
2.1	Граничные (Даты)	Успешно
3.1	Исключительные	Успешно
3.2	Исключительные	Успешно

5. Сравнительный анализ реализаций

В рамках выполнения курсовой работы были разработаны две версии программного обеспечения для управления фитнес-клубом: консольное

приложение на языке C++ и приложение с графическим интерфейсом на языке C# (Windows Forms).

Пользовательский интерфейс и взаимодействие Консольная реализация (C++) использует текстовое меню. Взаимодействие происходит путем ввода цифровых команд.

- *Преимущества:* Минимальные системные требования, кроссплатформенность, простота реализации базовой логики.
- *Недостатки:* Отсутствие наглядности, высокий риск ошибки ввода (человеческий фактор), невозможность визуального представления расписания.

Реализация на C# предоставляет полноценный графический интерфейс (GUI).

- *Преимущества:* Интуитивно понятное управление, использование специализированных элементов (календарь, выпадающие списки), моментальная валидация данных, табличное представление информации.
- *Недостатки:* Привязка к платформе Windows (.NET Framework/.NET Core).

Управление данными и памятью В версии на C++ использовалось ручное управление памятью и стандартные контейнеры STL (std::vector). Данные хранились в оперативной памяти во время сеанса. В версии на C# использована автоматическая сборка мусора (Garbage Collection), что исключает утечки памяти. Реализована система персистентности: данные сериализуются в формат JSON, что позволяет сохранять состояние системы между запусками.

Обработка ошибок

В реализации на C++ защита от некорректного ввода реализована через проверку состояния потока std::cin. Если пользователь вводит строковые символы там, где ожидается число (например, в меню), программа фиксирует ошибку флага потока, очищает буфер ввода и запрашивает данные повторно. Также реализована логическая валидация дат: алгоритм проверяет корректность дня и месяца перед созданием заказа.

В реализации на C# используется комбинированный подход:

1. **Интерфейсные ограничения:** Элементы управления DateTimePicker (календарь) и ComboBox (выпадающий список) физически не позволяют ввести некорректный формат данных.

2. **Программная валидация:** Перед регистрацией проверяется заполненность всех полей (`string.IsNullOrEmpty`). При попытке дублирования логина срабатывает проверка по базе данных.
3. **Обработка исключений:** Работа с файловой системой (чтение/запись JSON) обернута в блоки `try-catch` для предотвращения аварийного закрытия программы в случае отсутствия или повреждения файлов сохранения.

Личный выбор

На основе проведённого сравнительного анализа двух реализаций программы для управления фитнес-клубом, мой личный выбор склоняется в пользу реализации на **C#** с использованием **Windows Forms**.

Этот выбор обусловлен совокупностью факторов, делающих данную технологию наиболее подходящей для решения поставленной прикладной задачи:

1. Наглядность и эргономика интерфейса Специфика работы менеджера фитнес-клуба требует одновременного контроля множества данных: расписания тренеров, списков клиентов и статусов заказов. Графический интерфейс **C#** позволяет реализовать табличное представление данных (`DataGridView`), что невозможно сделать качественно в консоли. Использование визуальных компонентов, таких как календарь (`DateTimePicker`) и выпадающие списки (`ComboBox`), исключает возможность ввода некорректных данных и значительно ускоряет работу оператора .

2. Безопасность и скорость разработки Язык **C#** предоставляет среду с автоматическим управлением памятью (`Garbage Collection`). Это освобождает разработчика от необходимости вручную следить за освобождением ресурсов, что характерно для **C++**. Такой подход исключает целый класс ошибок, связанных с утечками памяти и неправильной работой с указателями, позволяя сосредоточиться на реализации бизнес-логики приложения .

3. Работа с данными (Персистентность) В версии на **C#** благодаря мощным встроенным библиотекам **.NET** удалось легко реализовать механизм сериализации объектов в формат **JSON**. Это обеспечивает надежное сохранение всей базы данных (клиентов, тренеров, заказов) в структурированном и читаемом виде. В **C++** реализация аналогичного механизма потребовала бы написания сложного парсера или подключения сторонних библиотек.

4. Практическая применимость С точки зрения конечного пользователя (сотрудника рецепции фитнес-клуба), консольное приложение является устаревшим и неудобным инструментом. Графическое приложение на C# соответствует современным стандартам программного обеспечения, имеет низкий порог вхождения для обучения персонала и готово к реальному внедрению в бизнес-процессы .

Заключение

В ходе выполнения работы была спроектирована и разработана информационная система «FitManager» для автоматизации процессов фитнес-клуба.

В соответствии с поставленными задачами было выполнено:

1. Проведен анализ предметной области, выделены ключевые роли (Клиент, Тренер, Администратор) и сущности (Заказ, Расписание).
2. Разработана объектно-ориентированная архитектура системы.
3. Реализовано консольное приложение на языке C++, демонстрирующее использование множественного наследования, полиморфизма и композиции.
4. Создано графическое приложение на языке C# (Windows Forms), поддерживающее полный цикл работы с заказами и сохранение данных в файлы.
5. Проведено функциональное тестирование обеих версий, подтвердившее их работоспособность и устойчивость к ошибкам.

Выводы: Разработанная система позволяет эффективно решать задачи записи клиентов, управления расписанием тренеров и ведения учета заказов. Реализация на C# показала себя как более пригодная для коммерческого использования благодаря удобному интерфейсу и надежной системе хранения данных.

Возможные направления развития:

- Переход от хранения данных в файлах JSON к полноценной базе данных (SQL Server или PostgreSQL).
- Реализация клиент-серверной архитектуры для одновременной работы множества пользователей.
- Добавление модуля финансовой отчетности и аналитики для администратора.